

UNIVERSIDAD PEDAGÓGICA Y TECNOLÓGICA DE COLOMBIA

Sistemas Operativos
Primer 50 %

Análisis y corrección de problemas de conurrencia en un sistema de despacho y logística sobre Linux

Documento técnico en formato IEEE

Repositorio oficial del proyecto

https://github.com/edwarfonseca/proyecto_final_so.git

Integrantes

Esteban Fonseca Jiménez
Marlon Andrés Delgado López
Juan Manuel Vianchá

Sogamoso, Boyacá, Colombia
2026

Análisis y corrección de problemas de concurrencia en un sistema de despacho y logística sobre Linux

Esteban Fonseca Jiménez, Marlon Andrés Delgado López y Juan Manuel Vianchá
 Universidad Pedagógica y Tecnológica de Colombia (UPTC)
 Sistemas Operativos - Primer 50 %

Resumen

Este artículo presenta el diseño, implementación, instrumentación y evaluación de un sistema concurrente de despacho y logística desarrollado en Python sobre Linux. El sistema combina procesos e hilos para modelar generadores de solicitudes, trabajadores de despacho, un taller de inspección y un monitor del estado del sistema. La implementación se diseñó para reproducir y estudiar tres familias de problemas de sistemas operativos: condición de carrera al asignar vehículos desde memoria compartida, interbloqueo por adquisición de vehículos y andenes en órdenes opuestos, y consumo elevado de CPU ocasionado tanto por espera activa como por una carga CPU-bound limitada por el Global Interpreter Lock (GIL) de CPython. La solución final incorpora una cola productor-consumidor acotada basada en semáforos, exclusión mutua para la sección crítica de asignación, un semáforo contador de vehículos libres, orden global de adquisición de recursos y, como alternativas, tiempo límite y detección con recuperación. La batería consolidada comprende 63 ejecuciones reales con semilla fija. Los resultados muestran 0 dobles asignaciones en las ejecuciones corregidas, 0 de 6 interbloqueos en las estrategias correctivas de la batería, y una reducción de 25.6 s a 0.26 s de CPU en el escenario de espera activa frente a espera bloqueante.

Index Terms

sistemas operativos, concurrencia, procesos, hilos, condición de carrera, interbloqueo, sincronización, GIL, Linux, memoria compartida

DESCRIPCIÓN DEL PROBLEMA

El proyecto modela un centro de despacho de una empresa de transporte que recibe solicitudes de entrega. Cada solicitud debe pasar, conceptualmente, por la secuencia *recibida* → *en cola* → *ruta planificada* → *vehículo asignado* → *cargue en andén* → *entrega* → *registro final*. La dificultad no está en implementar el flujo secuencial, sino en hacerlo bajo alta concurrencia y manteniendo invariantes de recursos.

La bitácora técnica define tres síntomas principales. Primero, dos solicitudes pueden observar simultáneamente un vehículo como libre y después marcarlo como ocupado, produciendo una asignación doble. Este patrón es una carrera *check-then-act* (también descrita como TOCTOU), porque la observación del estado y la modificación posterior no forman una operación indivisible. Segundo, dos operaciones diferentes pueden retener recursos y esperar el recurso retenido por la otra: el cargue solicita vehículo y luego andén, mientras la inspección solicita andén y luego vehículo. Tercero, el consumo de CPU aumenta por dos mecanismos diferentes: espera activa al no encontrar vehículo y una carga de cálculo de rutas que es CPU-bound.

La arquitectura también debía permitir reproducir fallos de forma controlada, observarlos desde el sistema operativo y comparar la situación anterior con la corregida usando la misma carga. Para ello, el programa conserva los modos problemáticos mediante parámetros de ejecución en lugar de eliminar el código con el fallo.

El entorno experimental registrado en la bitácora fue Linux (Arch), kernel 7.1.8-arch1-3, Python 3.14.7 sobre CPython con GIL activo, y 4 CPU lógicas. Para los experimentos de CPU y memoria se identificó el hardware Intel Core i3-10110U con 2 núcleos físicos y 4 hilos lógicos mediante Hyper-Threading. Estas características son relevantes porque el paralelismo observado está limitado por el hardware físico y por el modelo de ejecución de CPython.

La implementación, scripts de ejecución y materiales del proyecto están disponibles en [12].

https://github.com/edwarfonseca/proyecto_final_so.git

Cuadro I
SÍNTOMAS, CAUSAS Y TRATAMIENTO TÉCNICO

Síntoma	Causa de SO	Corrección implementada	Evidencia principal
Dos solicitudes sobre el mismo vehículo	Carrera <i>check-then-act</i> en memoria compartida	Mutex sobre buscar+marcar y semáforo contador de vehículos libres	Fases 3–4, batería Fase 8
Despachos que quedan esperando	Interbloqueo por órdenes opuestos vehículo→andén y andén→vehículo; también se estudió inanición tras la caída de un consumidor	Orden global de recursos; alternativas <code>timeout</code> y detección+recuperación; cola propia con semáforos	Fase 5, hallazgo H3
CPU elevada	Espera activa y cálculo CPU-bound afectado por el GIL	Semáforo bloqueante para la espera; procesos para explotar paralelismo real de CPU	Fases 4 y 6

OBJETIVO

Objetivo general

Construir y evaluar un sistema concurrente de despacho y logística capaz de reproducir, observar, explicar y corregir problemas clásicos de sistemas operativos relacionados con procesos, hilos, recursos compartidos, condiciones de carrera, interbloqueos y consumo de CPU/memoria.

Objetivos específicos

1. Implementar un proceso principal que administre trabajadores, generadores, taller, monitoreo, muestreo y cierre ordenado.
2. Combinar procesos para paralelismo de CPU y aislamiento de fallos con hilos para las actividades predominantemente bloqueantes o de espera.
3. Implementar una cola productor-consumidor acotada y memoria compartida real para la flota de vehículos.
4. Reproducir una condición de carrera sobre la asignación de vehículos y demostrar que el GIL no constituye un mecanismo de sincronización de alto nivel.
5. Corregir la carrera mediante exclusión mutua y un semáforo contador, y mantener una versión insegura seleccionable para la comparación experimental.
6. Provocar el interbloqueo vehículo-andén, identificar sus cuatro condiciones de Coffman y evaluar prevención por orden, prevención por tiempo límite y detección con recuperación [1].
7. Medir CPU, memoria, cambios de contexto y estados del kernel a través de `/proc`, `ps`, `pstree` y `top`, apoyándose además en `time.thread_time()` y `getrusage()` [2]–[7].

ARQUITECTURA DE LA SOLUCIÓN

La solución final es híbrida. El proceso principal `centro_despacho` administra el sistema. Dentro de él operan hilos generadores que producen solicitudes, un vigilante que construye el grafo de espera, un monitor que registra el estado lógico de la operación y un muestreador que observa CPU y memoria del árbol de procesos desde `/proc`. Cada proceso trabajador crea T hilos despachadores consumidores. Además, existe el proceso `taller`, cuyos hilos inspectores compiten por los mismos recursos físicos para materializar el escenario de interbloqueo.

La selección de procesos para trabajo CPU-bound se justifica porque, en CPython con GIL, los hilos de un mismo proceso no ejecutan bytecode Python en paralelo; la documentación de Python recomienda `multiprocessing` para aprovechar máquinas multinúcleo en cargas CPU-bound, mientras que `threading` es apropiado para tareas con espera o E/S [8]. Los procesos se crean explícitamente con `fork` en el experimento, dado que en Python 3.14 el método predeterminado en POSIX cambió a `forkserver`; la bitácora fija `fork` para mantener la jerarquía PID/PPID esperada [9].

Cuadro II
PROCESOS E HILOS DE LA SOLUCIÓN FINAL

Proceso	Hilo	Rol	Espera típica observada
centro_despacho	MainThread	Supervisa hijos, recoge resultados, envía centinelas y coordina cierre	poll_schedule_timeout
	generador-i	Productores, generación en ráfagas sincronizadas	hrtimer_nanosleep o futex_do_wait
	vigilante	Construye grafo de espera y detecta ciclos	futex_do_wait
	monitor	Registra estado y alerta sin progreso	futex_do_wait
	muestreador	Mide CPU y memoria desde /proc	futex_do_wait
trabajador-w	MainThread	Crea despachadores y vigila orfandad	hrtimer_nanosleep
	despachador-w-t	Consume, calcula ruta, asigna vehículo, carga y entrega	R, futex_do_wait o hrtimer_nanosleep
	QueueFeederThread	Hilo auxiliar de la cola de resultados	futex_do_wait
taller	MainThread	Crea inspectores y vigila orfandad	hrtimer_nanosleep
	inspector-i	Inspecciona andén y vehículo	hrtimer_nanosleep o futex_do_wait

Flujo de una solicitud

Los generadores depositan solicitudes en una cola acotada. Los despachadores extraen una solicitud, planifican su ruta y solicitan un vehículo. La planificación se hace antes de adquirir el vehículo para no prolongar ninguna sección crítica. Una vez obtenido el vehículo, el despacho utiliza los recursos físicos según la estrategia de interbloqueo configurada y finalmente libera el vehículo y publica el resultado.

La cola propia implementa el patrón productor-consumidor con semáforos que representan posiciones vacías y llenas y mutexes separados para lectura y escritura. La capacidad K introduce *backpressure*: si el búfer está lleno, el productor espera; si está vacío, el consumidor espera. La implementación final se escogió después de detectar un caso en el que un `multiprocessing.Queue` podía quedar sin progreso si un participante moría reteniendo un lock interno.

PROCESOS E HILOS

El sistema utiliza una jerarquía observable mediante PID, PPID y TID. En una configuración representativa de 2 trabajadores por 3 hilos despachadores, 2 generadores y 1 inspector, participan cuatro procesos principales del modelo de aplicación y 18 hilos contabilizados, sin considerar los hilos auxiliares creados internamente por bibliotecas.

En Linux, `/proc/<pid>/task/` contiene un subdirectorío por hilo del proceso, identificado por su TID, lo que permite observar directamente la correspondencia entre los hilos de Python y las tareas planificables del kernel [2]. La bitácora utiliza esta capacidad junto con `ps -L`, `ps -eLf`, `top -H` y `pstree -p -t`.

La estrategia de terminación también forma parte de la arquitectura. El proceso principal instala manejadores para `SIGINT` y `SIGTERM`; los trabajadores ignoran `SIGINT` y esperan instrucciones del principal. El cierre normal usa centinelas y `join()`; el cierre forzado escala de `SIGTERM` a `SIGKILL`. Cuando un trabajador termina, el principal lo recoge con `waitpid/join` para evitar zombis. La detección de orfandad se basa en observar el cambio de PPID.

RECURSOS COMPARTIDOS

La solución comparte estado entre procesos sólo en los puntos en los que es necesario. Esto permite demostrar de forma auténtica el problema de concurrencia, en lugar de ocultarlo detrás de un proceso servidor. La flota se almacena en un `RawArray` de memoria compartida; los estados representan vehículo libre con 0 y vehículo asignado con el identificador de la solicitud.

La evidencia de memoria compartida se obtuvo leyendo `/proc/<pid>/maps`: el mismo inodo de un objeto de `/dev/shm/pym-*` aparecía en múltiples procesos. Esto es coherente con la semántica

Cuadro III
RECURSOS COMPARTIDOS Y MECANISMOS DE PROTECCIÓN

Recurso	Representación	Compartido entre	Protección / sincronización
Cola de solicitudes	Pipe del kernel + semáforos POSIX	Generadores y despachadores	vacíos, llenos, mutex_lectura, mutex_escritura
Flota estado[V]	RawArray en memoria compartida	Todos los despachadores	Mutex para buscar+marcar+liberar y BoundedSemaphore(V) para cantidad disponible
Vehículos y andenes físicos	Un Lock por recurso	Despachadores e inspectores	Orden global; alternativas timeout y detección/recuperación
Registro de recursos	Arreglos compartidos de dueño/espera	Despachadores, inspectores y vigilante	Un lock para tomar fotografías consistentes
Contadores	RawArray/Value	Procesadores de estadísticas y despachadores	Lock alrededor de cada secuencia leer-modificar-escribir
Cola de resultados	multiprocessing.Queue	Trabajadores y principal	Primitivas internas; un solo lector en el principal
Indicador de parada	RawValue de un byte	Principal y todos los hijos/hilos	Un único escritor; lectura por los demás
Historial de trazas	OrderedDict en el heap del trabajador	Hilos del mismo trabajador	threading.Lock

de memoria compartida y con la recomendación de Python de heredar o pasar recursos compartidos de forma explícita entre procesos [9]. Los semáforos y locks observados en `/dev/shm/sem.*` representan primitivas utilizadas para sincronización entre procesos.

PROBLEMA DE CONCURRENCIA

El fallo principal aparece en la asignación de vehículos. La versión insegura puede simplificarse como sigue:

Listing 1. Secuencia insegura de asignación de vehículo

```
def buscar_y_marcar(id_sol):
    v = primer_libre()          # (1) check
    if v is not None:
        time.sleep(ventana)     # ventana artificial; puede ser 0
        estado[v] = id_sol      # (2) act
    return v
```

La operación parece sencilla, pero `primer_libre()` y la escritura en `estado[v]` están separadas. Dos despachadores pueden leer el mismo valor 0 antes de que cualquiera escriba el identificador de la solicitud. En las pruebas con 24 solicitudes, la versión insegura produjo 18 dobles asignaciones en una ejecución demostrativa; en la batería consolidada, entre 63 % y 81 % de las asignaciones quedaron en conflicto para las cargas de 10 a 200 solicitudes. La auditoría y la sonda independiente coincidieron en los casos reproducidos.

El fenómeno ocurre tanto entre procesos como entre hilos. Con una ventana artificial de 1 ms, seis procesos/hilos concurrentes dispararon el fallo de forma reproducible. Sin ventana artificial, la carrera seguía presente: para la configuración de seis hilos en un proceso apareció en 0/10 ejecuciones, pero con seis procesos apareció en 10/10, y la configuración híbrida también presentó fallos. La ausencia de fallo en una ejecución no constituye evidencia de corrección.

Un punto importante es el GIL. En una construcción estándar de CPython, el GIL restringe a un hilo por proceso a ejecutar código Python a la vez, pero no vuelve atómica una secuencia lógica como *leer, verificar, dormir y escribir*. La documentación de CPython indica explícitamente que el GIL no sustituye los locks necesarios para garantizar thread-safety en código Python [8], [10]. Además, entre procesos no existe un GIL compartido.

La magnitud del problema se observa también físicamente: con 3 vehículos, la versión insegura llegó a registrar hasta 6 entregas simultáneas. El programa corregido nunca superó 3 entregas simultáneas. Por tanto, la mayor tasa aparente de solicitudes/s de la versión insegura no representa una mejora de rendimiento, sino el procesamiento de un estado imposible.

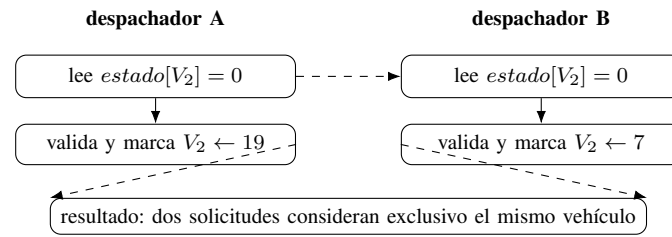


Figura 1. Intercalado que materializa la condición de carrera *check-then-act*.

MECANISMO DE SINCRONIZACIÓN

La corrección separa dos responsabilidades que no debe asumir una sola primitiva. El mutex determina *qué vehículo* toma un despachador; el semáforo contador determina *cuántos despachadores* pueden disponer simultáneamente de un vehículo.

Listing 2. Asignación corregida y sección crítica fina

```

def asignar(id_sol):
    while not disponibles.acquire(timeout=0.5):
        if detener.value:
            return None

    with mutex:
        v = primer_libre()      # check
        if v is not None:
            estado[v] = id_sol  # act

    if ventana:
        time.sleep(ventana)     # fuera del mutex
    return v

def liberar(v, id_sol):
    with mutex:
        estado[v] = 0
    disponibles.release()

```

La propiedad clave es que la búsqueda y el marcado ocurren dentro de una sola sección crítica. Quien llega segundo no puede observar un vehículo como libre mientras otro flujo se encuentra entre la lectura y la escritura. La sección crítica es deliberadamente fina: la validación artificial de 50 ms se mantiene fuera del mutex después de que el vehículo ya está reservado.

Las pruebas muestran que el mutex y el semáforo tienen efectos distintos. Con modo inseguro y espera bloqueante, todavía aparecieron dobles asignaciones en 8 de 10 ejecuciones; por tanto, el semáforo por sí solo limita la cantidad de vehículos usados, pero no decide qué vehículo concreto corresponde a cada hilo. De forma inversa, con modo seguro y espera activa la corrección se obtuvo, pero se produjeron alrededor de 2372 búsquedas fallidas por ejecución. La combinación de mutex + semáforo bloqueante elimina ambos problemas.

La implementación utiliza primitivas de `multiprocessing` que sincronizan también entre procesos. La documentación de Python indica que `multiprocessing.Lock` bloquea adquisiciones posteriores hasta la liberación del lock y que `BoundedSemaphore` es apropiado para controlar una cantidad acotada de recursos [9]. En Linux, cuando la espera llega al kernel puede observarse `futex_do_wait`; el mecanismo `futex` está diseñado precisamente para implementar esperas y despertares sobre estados de sincronización compartidos [11].

El resultado experimental de la corrección, con la misma carga y semilla, fue:

El incremento del tiempo total no es un costo significativo del mutex: la espera media por mutex fue de 0.01 ms. El aumento se debe a que el sistema corregido respeta la capacidad real de la flota y ya no permite seis entregas simultáneas sobre tres vehículos.

ANÁLISIS DEL INTERBLOQUEO, SI APLICA

El interbloqueo aplica de forma directa. Se diseñaron dos operaciones:

Cuadro IV
ANTES Y DESPUÉS DE SINCRONIZAR LA FLOTA

Métrica	Antes	Después
Ejecuciones con doble asignación	10/10	0/10
Dobles asignaciones promedio	16.8	0
Registros inconsistentes	17.6	0
Máximo en ruta con 3 vehículos	6	3
Tiempo total (s)	2.61	4.89
Rendimiento (sol/s)	9.23	4.91
Espera por mutex (ms)	–	0.01
Código de salida	1	0

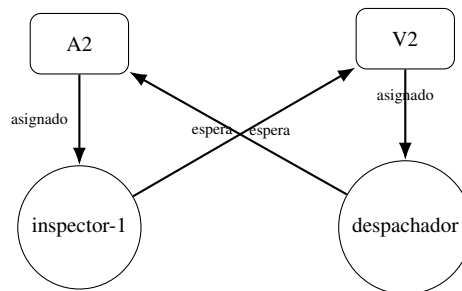


Figura 2. Ciclo de asignación de recursos que genera el interbloqueo vehículo-andén.

- **Cargue:** el despachador adquiere primero el vehículo y después el andén.
- **Inspección:** el inspector del taller adquiere primero el andén y después el vehículo.

Con un intercalado adverso, el inspector puede retener A2 y esperar V2, mientras el despachador retiene V2 y espera A2. El grafo de espera queda como

$$\text{inspector-1} \rightarrow \text{despachador-2-3} \rightarrow \text{inspector-1}.$$

Cuando cada recurso tiene una sola instancia, un ciclo de este tipo caracteriza el interbloqueo.

Las cuatro condiciones clásicas de Coffman se cumplen [1]. Hay exclusión mutua porque cada vehículo y cada andén es de uso exclusivo; hay retención y espera porque el primer recurso se conserva mientras se solicita el segundo; no existe expropiación automática de un lock; y existe espera circular debido a los órdenes opuestos.

El vigilante toma una fotografía consistente del registro de propietarios y esperas cada 0.5 s y busca ciclos. Para reducir falsos positivos de transiciones instantáneas, el ciclo debe aparecer en dos fotos consecutivas. La detección es lineal en el número de hilos porque cada hilo espera, como máximo, un recurso y por tanto cada nodo tiene como máximo una arista saliente.

Se probaron tres estrategias correctivas además del modo inseguro `sin_orden`. En la batería de la Fase 8, `sin_orden` produjo interbloqueo en 6/6 ejecuciones; `orden`, `timeout` y `deteccion` produjeron 0/6 ejecuciones sin resolver. En una prueba previa de diez ejecuciones por estrategia, las tres soluciones correctivas completaron 24/24 entregas por ejecución.

Evidencia del interbloqueo

Durante un caso real, `ps -L` mostró a `inspector-1` y `despachador-2-3` en estado S, con `wchan=futex_do_wait`, 0 CPU y 0 cambios de contexto durante un intervalo de 2 s. El monitor dejó de observar nuevas entregas. Esto permite distinguir interbloqueo de lentitud: una espera ordinaria puede ser transitoria; el ciclo persistente acompañado de ausencia de progreso es el indicio consistente con deadlock.

Cuadro V
CONDICIONES DE COFFMAN Y ESTRATEGIAS DE TRATAMIENTO

Condición	Manifestación en el sistema	Tratamiento
Exclusión mutua	Un vehículo o un andén es utilizado por una sola operación a la vez	No se elimina, porque es una propiedad física del recurso
Retención y espera	El despachador retiene el vehículo mientras espera el andén; el inspector retiene el andén mientras espera el vehículo	timeout: libera el primero y reintenta
No expropiación	Ningún participante puede quitar un lock al dueño	deteccion: el vigilante elige una víctima y ordena abandono de sus recursos
Espera circular	Órdenes vehículo→andén y andén→vehículo	orden: todos los participantes adquieren recursos según un orden global

Cuadro VI
COMPARACIÓN DE ESTRATEGIAS DE INTERBLOQUEO

Estrategia	Bloqueos	Detectados/recup.	Reintentos	Tiempo 48 (s)
sin_orden	6/6	0/0	0	no termina
orden	0/6	0/0	0	10.64 ± 0.13
timeout	0/6	0/0	8.0–9.3	10.83 ± 0.29
deteccion	0/6	3.0–3.7/3.0–3.7	0	12.19 ± 0.95

PRUEBAS DE CPU Y MEMORIA

Prueba CPU-bound: fuerza bruta para la ruta

Cada solicitud puede contener P puntos y la ruta óptima se busca por fuerza bruta, evaluando $P!$ recorridos. En el equipo de pruebas, 7 puntos requirieron aproximadamente 3 ms de CPU, 8 puntos alrededor de 25 ms y 9 puntos alrededor de 220 ms por solicitud. Esta carga es intencional: es determinista, regulable y suficientemente pesada para hacer visible el comportamiento del planificador y del GIL.

La CPU de cada hilo se midió con `time.thread_time()`, que reporta el tiempo de CPU del hilo y no incluye el tiempo dormido; el proceso se complementó con `getrusage()` y con lecturas de `/proc/<pid>/stat` [3], [6], [7].

Cuadro VII
CPU: HILOS FRENTE A PROCESOS EN 24 RUTAS DE 9 PUNTOS

Procesos	Hilos	Tiempo (s)	Aceleración	CPU trabajadores / Real-CPU	Cambios invol.
1	1	5.31	1.00x	95 % / 1.00	250
1	2	5.41	0.98x	96 % / 1.86	113
1	4	5.35	0.99x	96 % / 3.50	184
1	8	5.41	0.98x	96 % / 5.34	283
2	1	3.52	1.51x	180 % / 1.03	1274
4	1	3.01	1.76x	334 % / 1.08	3460
8	1	2.87	1.85x	349 % / 2.00	6564
2	2	3.63	1.46x	186 % / 1.94	632
4	2	2.98	1.78x	337 % / 2.13	8317

Nota: “CPU trabajadores / Real-CPU” combina el porcentaje agregado de CPU y la razón tiempo de pared/CPU reportadas en la bitácora. La CPU total de rutas fue de 5.27–10.89 s según la configuración.

Con uno o varios hilos dentro de un solo proceso, la aceleración permanece alrededor de 1x y `top -H` muestra que sólo un hilo está en R a la vez. Con procesos, el CPU total puede superar el 300 % porque distintos procesos ejecutan en paralelo. El mejor resultado de la tabla es 1.85x con ocho procesos, pero la ganancia adicional sobre cuatro procesos es limitada por el hardware: el equipo tiene dos núcleos físicos y cuatro CPU lógicas. La misma bitácora verificó que la suma de longitudes de las rutas permaneció en 7690.4 en las configuraciones comparadas.

La documentación de CPython confirma que el GIL limita las ganancias de `threading` en cargas CPU-bound y recomienda procesos cuando se necesita aprovechar una máquina multinúcleo [8]. También

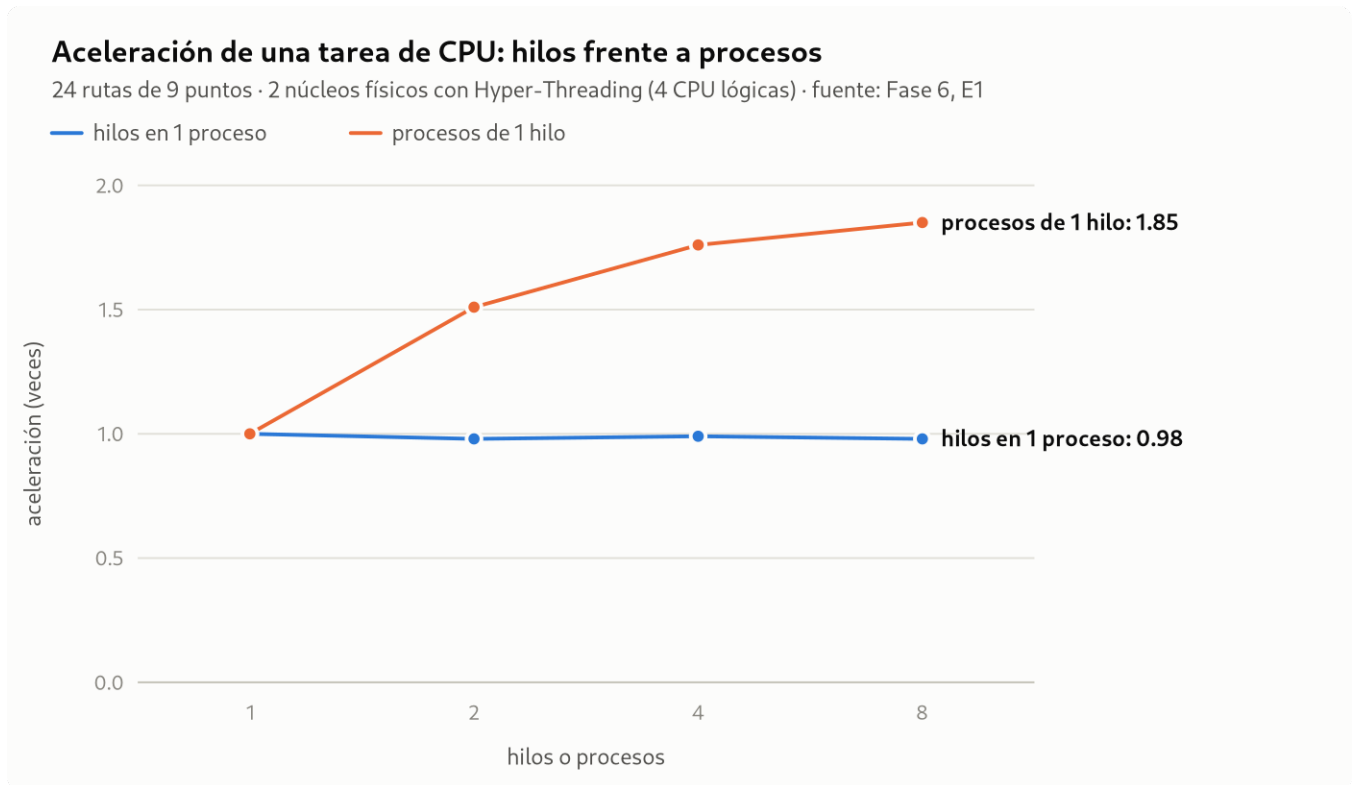


Figura 3. Aceleración observada en la carga CPU-bound al aumentar el paralelismo por hilos o por procesos.

Cuadro VIII
COSTO DE LA ESPERA ACTIVA

Modo	Tiempo (s)	CPU (s)	CPU med.	Cambios vol.	Sondeos
Bloqueante	14.80 ± 0.04	0.26	1.8 %	1571	0
Activa 5 ms	14.81 ± 0.15	2.91	19.3 %	49982	33064
Activa sin pausa	14.87 ± 0.19	25.62	169 %	4223135	2025958

indica que el GIL se libera alrededor de operaciones bloqueantes, lo cual explica por qué los hilos siguen siendo útiles en el resto del sistema [10].

Prueba de espera activa

Con 16 despachadores, 2 vehículos y 48 solicitudes, se compararon la espera bloqueante mediante semáforo y dos variantes de espera activa.

El tiempo total apenas cambia porque lo determina la disponibilidad de los dos vehículos. Lo que cambia es el trabajo improductivo: el modo activo sin pausa consume casi 99 veces más CPU que el bloqueo por semáforo para terminar prácticamente en el mismo tiempo. En el modo bloqueante los hilos sin vehículo aparecen dormidos en un `futex`; en el modo activo se despiertan periódicamente para volver a comprobar la flota.

Prueba de memoria

Cada entrega puede almacenar una traza GPS de 256 KB. Al usar `-historial 0`, las trazas no se eliminan; con `-historial 20` sólo se conservan las 20 últimas. El muestreador lee `RSS`, `PSS` y `Private_Dirty` desde `/proc` cada 0.25–0.5 s. En `/proc/<pid>/smaps`, `RSS` es la cantidad residente en RAM de cada mapeo y `PSS` distribuye la participación proporcional de páginas compartidas

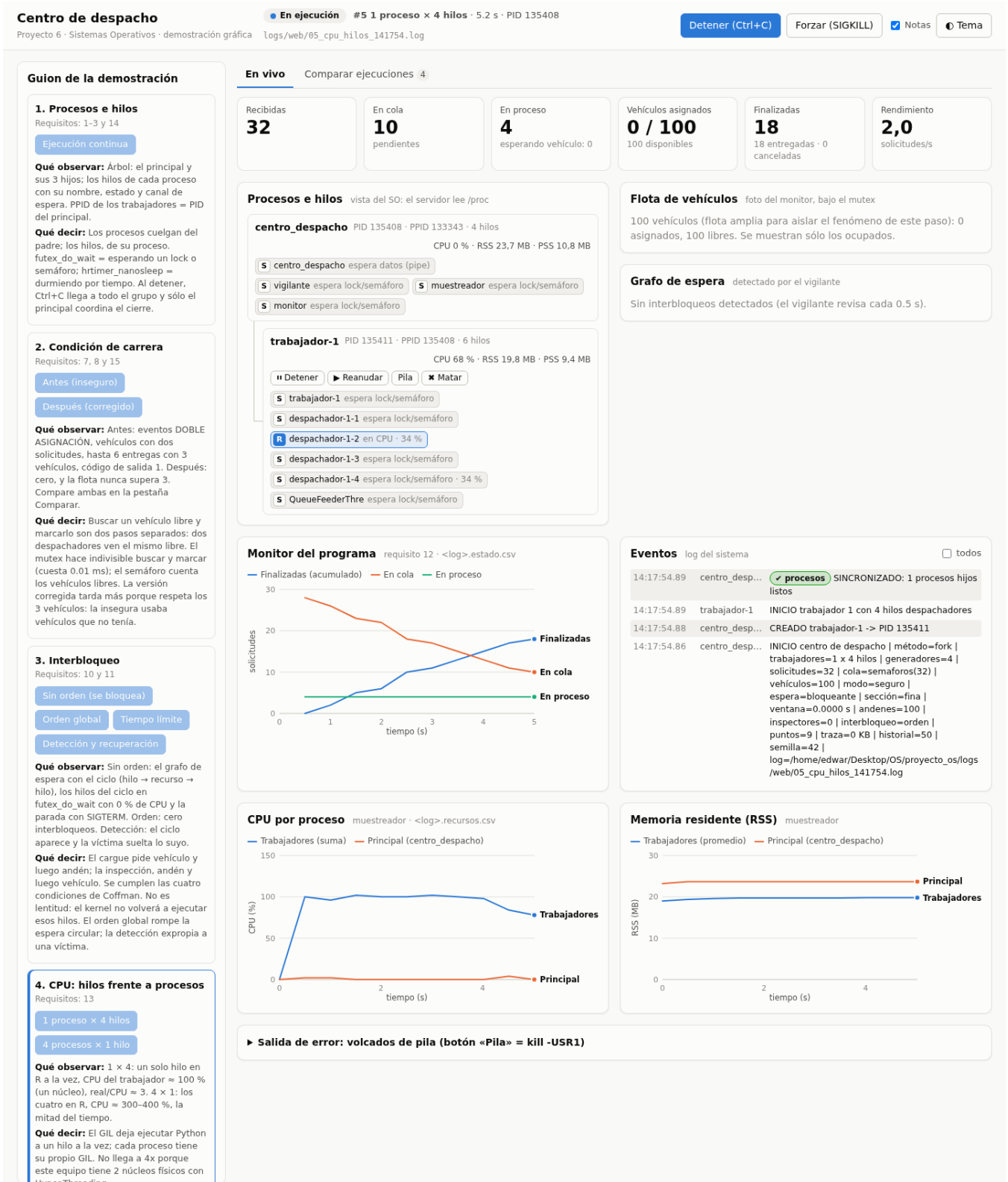


Figura 4. Captura de la evidencia del GIL: con un proceso y cuatro hilos, sólo un hilo permanece en estado de ejecución para la tarea CPU-bound.

[5]. RSS, PSS y VSZ no representan lo mismo; por eso la bitácora utiliza PSS y memoria privada además de RSS.

Sin límite, el RSS creció de forma aproximadamente lineal con las entregas; con historial acotado, la

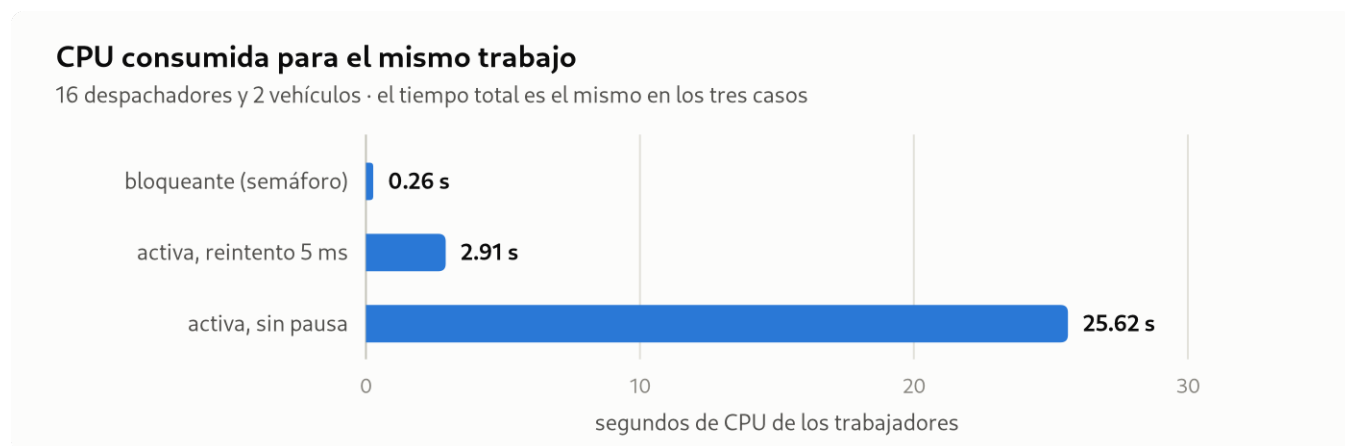


Figura 5. CPU consumida según el mecanismo de espera para una carga equivalente.

Cuadro IX
MEMORIA CON HISTORIAL SIN LÍMITE FRENTE A HISTORIAL ACOTADO

Métrica final por trabajador	Sin límite	Acotado a 20
Trazas conservadas	100	20
Trazas descartadas	0	81
RSS (MB)	46.2	27.0
Private Dirty (MB)	29.6	10.3
PSS total de picos (MB)	81.9	45.4

serie se estabilizó alrededor de 27 MB. Este comportamiento demuestra el efecto de retener objetos de tamaño fijo en un historial que crece sin límite. El aumento fue privado de cada trabajador: los procesos no comparten su heap de Python, mientras que los hilos del mismo proceso sí comparten el historial y por ello requieren `threading.Lock`.

EVIDENCIAS

Las evidencias se generaron mediante scripts reproducibles y quedaron almacenadas en los directorios correspondientes a las fases 1 a 8, además de las capturas de la Fase 10. El objetivo no fue sólo demostrar que “el programa corre”, sino relacionar el síntoma observado con una causa verificable en el sistema operativo.

Las herramientas del SO aportan una segunda fuente de evidencia. `ps` y `pstree` muestran jerarquía, estados y cantidad de hilos; `top -H` muestra CPU por hilo; `/proc/<pid>/task/` permite inspeccionar cada TID; `/proc/<pid>/stat` aporta tiempo de CPU y RSS; `/proc/<pid>/status` expone cambios de contexto; y `/proc/<pid>/smaps` permite analizar RSS, PSS y memoria privada [2]–[5]. Los manuales de Linux definen además `futex` como una infraestructura para esperas y despertares sobre memoria compartida [11].

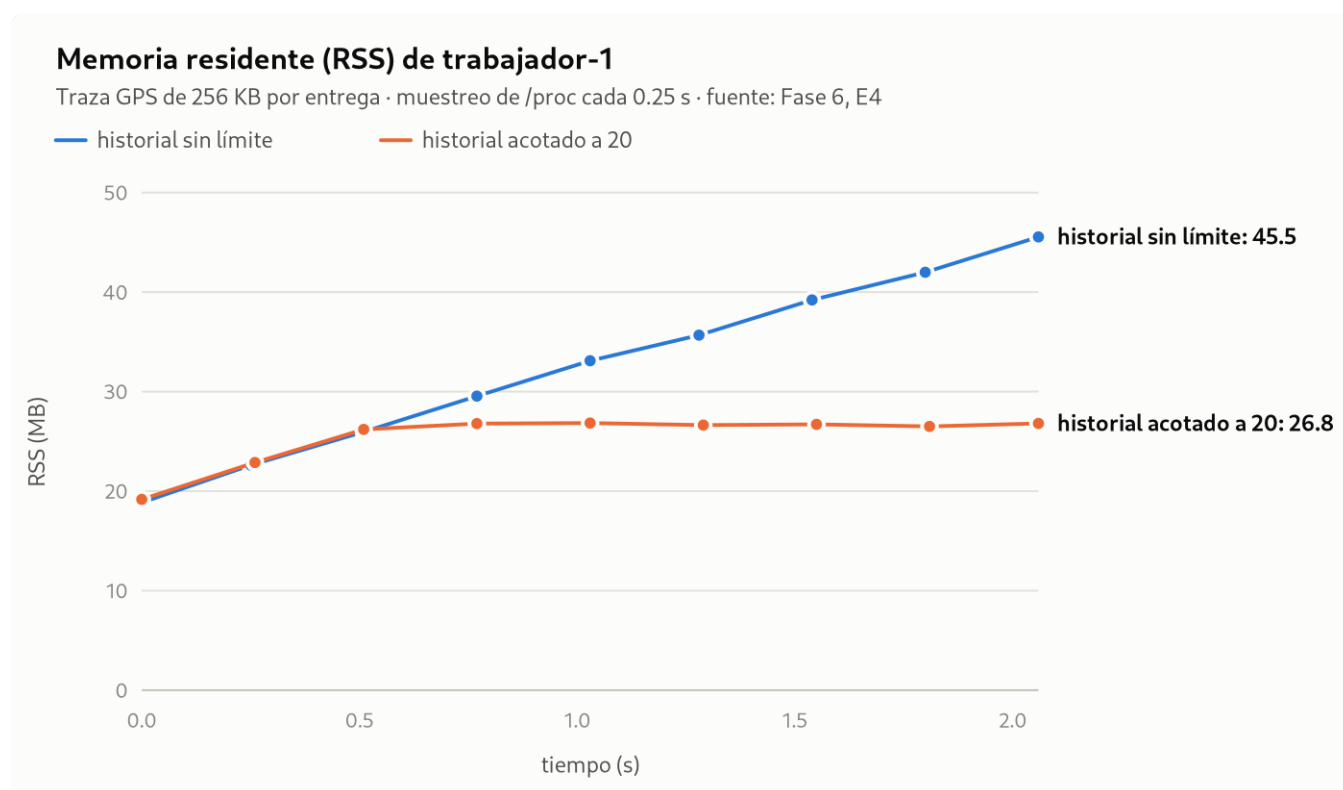


Figura 6. Crecimiento del RSS con historial sin límite y estabilización con historial acotado.

Cuadro X
MATRIZ COMPACTA DE EVIDENCIAS

Fenómeno	Evidencia	Relación con la conclusión
Procesos e hilos	fase1/e1_observacion.txt, fase2/e2_hilos.txt, 01_procesos_e_hilos.png	captura PID/PPID/TID, estados y roles reales
Carrera	fase3/e1_dobles.txt, fase3/e1_cronologia_V2.txt, 02_carrera_en_vivo.png	captura Dos flujos marcan el mismo vehículo; salida 1 y auditoría coincidente
Sincronización	fase4/e1_antes_despues.txt, fase4/e3_mecanismos.txt	0 dobles asignaciones en el modo corregido
Interbloqueo	fase5/e1_observacion.txt, fase5/e1_cronologia.txt, 05_interbloqueo.png	captura Ciclo en grafo de espera + futex_do_wait + 0 CPU + ausencia de progreso
CPU	fase6/e1_cpu_hilos_procesos.txt, fase6/e2_top_wl_t4.txt	GIL, aceleración por procesos y cambios de contexto
Memoria	fase6/e4_historial_*.recursos.csv, fase6/e4_memoria.txt	Crecimiento lineal de RSS y estabilización con límite
Comparación	fase8/resultados.csv, fase8/resumen.md, gráficas G1–G8	Consolidación de 63 ejecuciones

Centro de despacho

En ejecución

#1 Ejecución continua · 5.1 s · PID 133394

Detener (Ctrl+C)

Forzar (SIGKILL)

Notas

Tema

Proyecto 6 · Sistemas Operativos · demostración gráfica

logs/web/01_procesos_141711.log

Guion de la demostración

1. Procesos e hilos

Requisitos: 1-3 y 14

Ejecución continua

Qué observar: Árbol: el principal y sus 3 hijos; los hilos de cada proceso con su nombre, estado y canal de espera. PPID de los trabajadores = PID del principal.

Qué decir: Los procesos cuelgan del padre; los hilos, de su proceso. `futex_do_wait` = esperando un lock o semáforo; `hrtimer_nanosleep` = durmiendo por tiempo. Al detener, Ctrl+C llega a todo el grupo y sólo el principal coordina el cierre.

2. Condición de carrera

Requisitos: 7, 8 y 15

Antes (inseguro)

Después (corregido)

Qué observar: Antes: eventos DOBLE ASIGNACIÓN, vehículos con dos solicitudes, hasta 6 entregas con 3 vehículos, código de salida 1. Después: cero, y la flota nunca supera 3. Compare ambas en la pestaña Comparar.

Qué decir: Buscar un vehículo libre y marcarlo son dos pasos separados: dos despachadores ven el mismo libre. El mutex hace indivisible buscar y marcar (cuesta 0.01 ms); el semáforo cuenta los vehículos libres. La versión corregida tarda más porque respeta los 3 vehículos; la insegura usaba vehículos que no tenía.

3. Interbloqueo

Requisitos: 10 y 11

Sin orden (se bloquea)

Orden global

Tiempo límite

Detección y recuperación

Qué observar: Sin orden: el grafo de espera con el ciclo (hilo → recurso → hilo), los hilos del ciclo en `futex_do_wait` con 0 % de CPU y la parada con SIGTERM. Orden: cero interbloques. Detección: el ciclo aparece y la víctima suelta lo suyo.

Qué decir: El cargue pide vehículo y luego andén; la inspección, andén y luego vehículo. Se cumplen las cuatro condiciones de Coffman. No es

En vivo

Comparar ejecuciones

Recibidas

36

En cola

8

pendientes

En proceso

6

esperando vehículo: 3

Vehículos asignados

3 / 3

0 disponibles

Finalizadas

22

22 entregadas · 0 canceladas

Rendimiento

6,0

solicitudes/s

Procesos e hilos

vista del SO: el servidor lee /proc

centro_despacho

PID 133394 · PPID 133343 · 6 hilos

CPU 0 % · RSS 23,4 MB · PSS 8,0 MB

S centro_despacho espera datos (pipe)

S vigilante espera lock/semáforo

S muestreador espera lock/semáforo

S generador-1 duerme (tiempo)

S generador-2 espera lock/semáforo

S monitor espera lock/semáforo

trabajador-1

PID 133397 · PPID 133394 · 5 hilos

CPU 0 % · RSS 20,5 MB · PSS 7,1 MB

Detener

Reanudar

Pila

Matar

S trabajador-1 duerme (tiempo)

S despachador-1-1 espera lock/semáforo

S despachador-1-2 espera lock/semáforo

S despachador-1-3 espera lock/semáforo

S QueueFeederThre espera lock/semáforo

trabajador-2

PID 133398 · PPID 133394 · 5 hilos

CPU 0 % · RSS 20,5 MB · PSS 7,1 MB

Detener

Reanudar

Pila

Matar

S trabajador-2 duerme (tiempo)

S despachador-2-1 duerme (tiempo)

S despachador-2-2 duerme (tiempo)

S despachador-2-3 duerme (tiempo)

S QueueFeederThre espera lock/semáforo

taller

PID 133400 · PPID 133394 · 2 hilos

CPU 0 % · RSS 18,9 MB · PSS 5,8 MB

Detener

Reanudar

Pila

Matar

S taller duerme (tiempo)

S inspector-1 espera lock/semáforo

Flota de vehículos

foto del monitor, bajo el mutex

V1

Solicitud 1000011

V2

Solicitud 1000010

V3

Solicitud 1000013

Grafo de espera

detectado por el vigilante

Sin interbloques detectados (el vigilante revisa cada 0.5 s).

Monitor del programa

requisito 12 · <log>.estado.csv

Finalizadas (acumulado)

En cola

En proceso

30

20

Eventos

log del sistema

todos

procesos

14:17:11.34

centro_desp...

✓ procesos

SINCRONIZADO: 3 procesos hijos listos

14:17:11.34

trabajador-2

INICIO trabajador 2 con 3 hilos despachadores

14:17:11.34

taller

INICIO taller con 1 hilos inspectores

Figura 7. Evidencia del árbol de procesos e hilos observada desde el frontend.

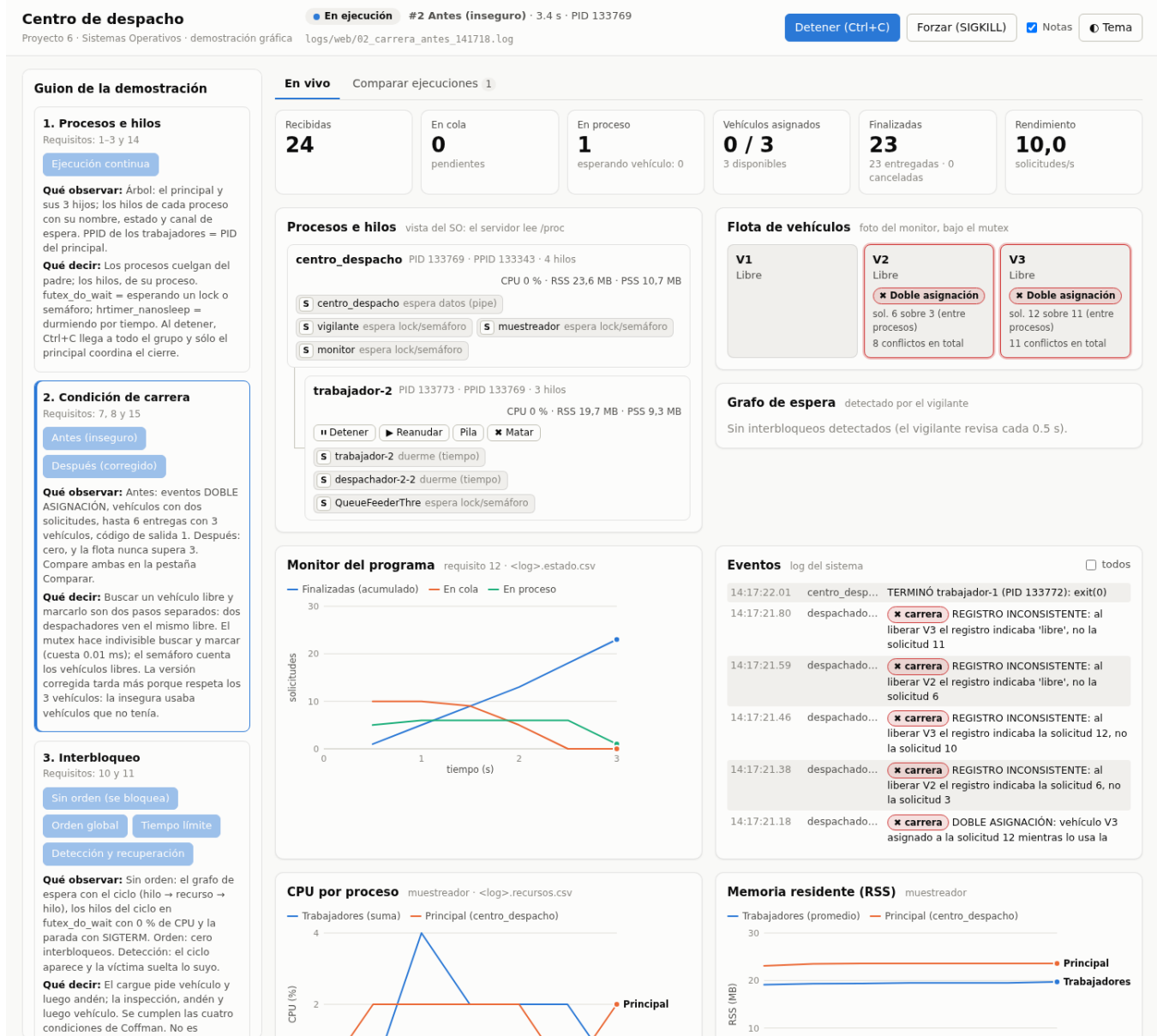


Figura 8. Evidencia de la doble asignación durante la ejecución insegura.

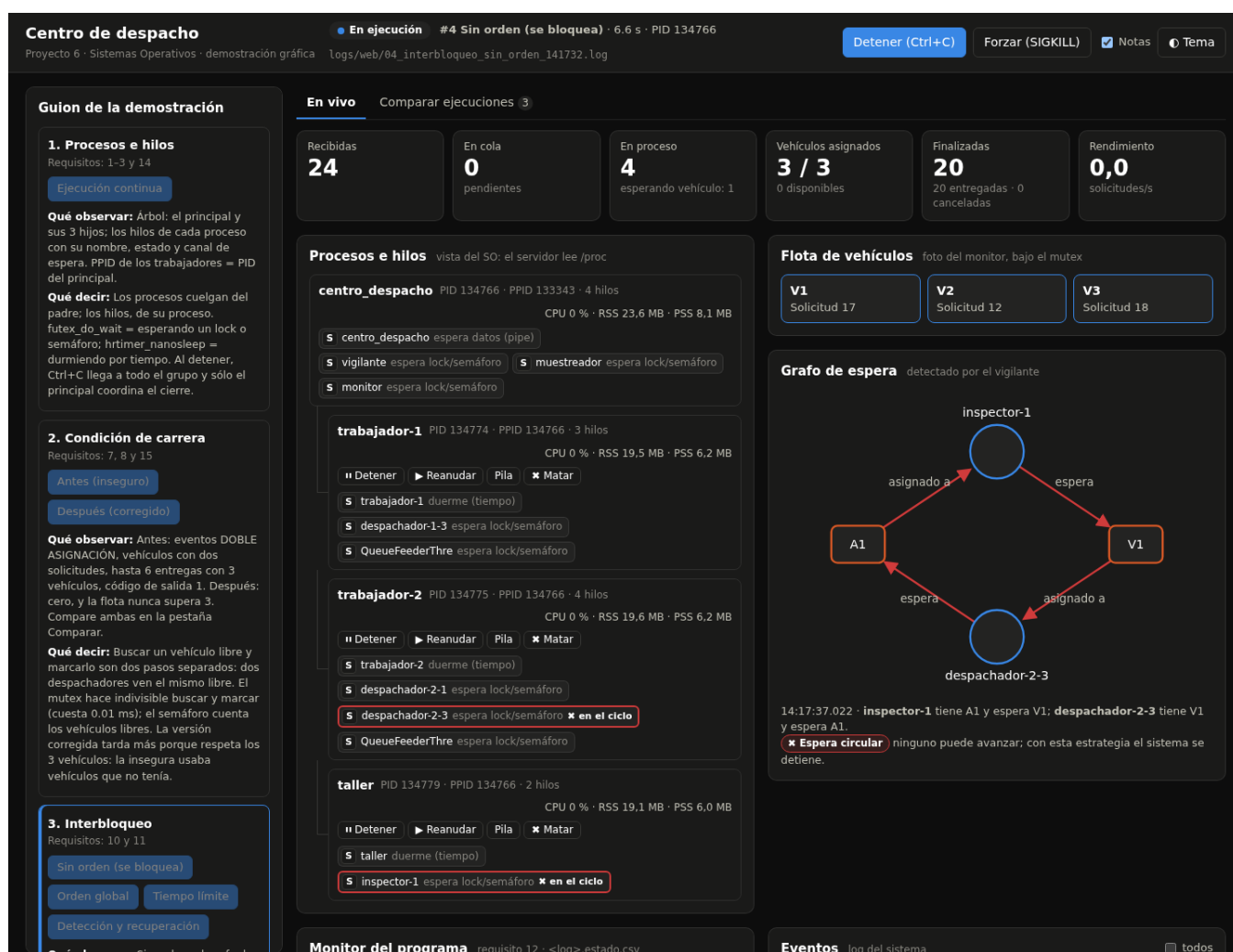


Figura 9. Evidencia del grafo de espera y el ciclo de interbloqueo.

Cuadro XI
COMPARACIÓN CONSOLIDADA DE LOS SÍNTOMAS DEL SISTEMA

Síntoma	Causa	Corrección	Antes	Después
Asignación duplicada	Carrera sobre estado [V]	Mutex + semáforo contador	63–81 % en conflicto; hasta 6/3 simultáneas	0 conflictos; máximo 3/3
Interbloqueo	Ordenes opuestos sobre vehículo/andén	Orden global; timeout; detección+recuperación	6/6 con <code>sin_orden</code>	0/6 con cada estrategia correctiva
Espera activa	Sondeo periódico sin vehículo	Semáforo bloqueante	25.62 s CPU; 169 %	0.26 s CPU; 1.8 %
CPU-bound	GIL serializa hilos del proceso	Procesos para la parte CPU	0.98x con 8 hilos	1.76x con 4 procesos; 1.85x con 8
Memoria	Historial no acotado	Límite de trazas retenidas	RSS 46.2 MB	RSS 27.0 MB

COMPARACIÓN ANTES/DESPUÉS

La comparación experimental se diseñó con dos condiciones de control: misma semilla (42) y misma carga de solicitudes. Sólo se cambia el mecanismo que se desea evaluar. La batería global tuvo 21 configuraciones, cada una con tres repeticiones, para un total de 63 ejecuciones reales.

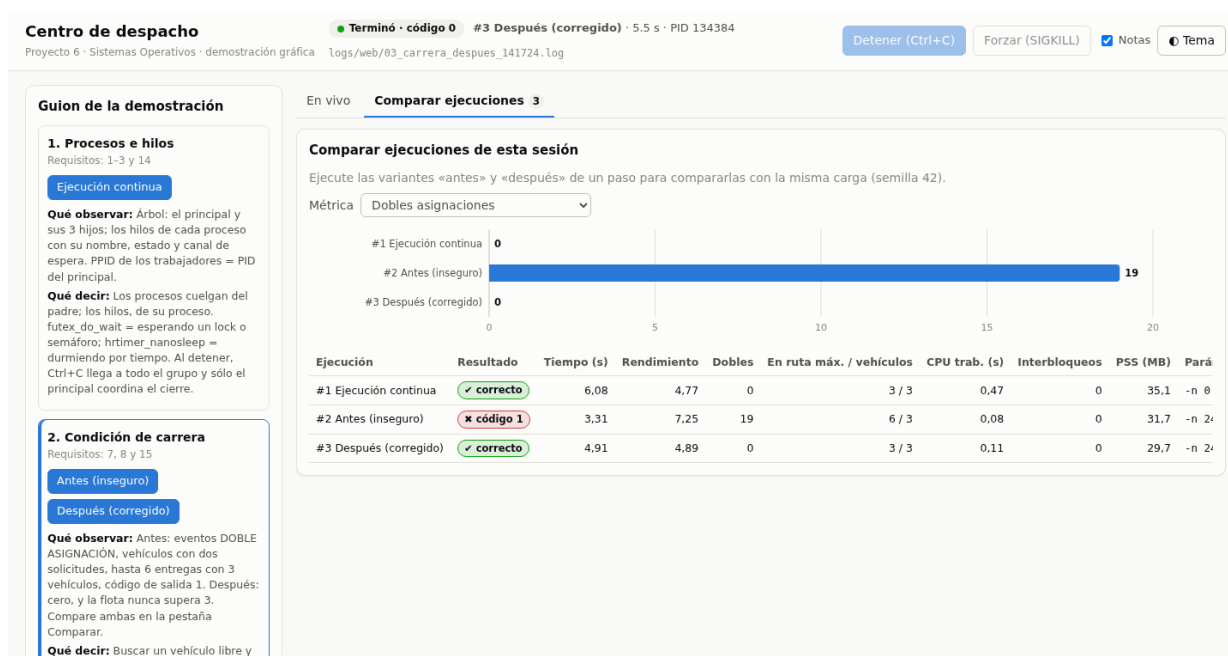


Figura 10. Vista de comparación antes/después de la interfaz de demostración.

Para la flota, la versión insegura alcanzó 8.19 solicitudes/s con 50 solicitudes y 8.56 solicitudes/s con 200. Estos valores son superiores al máximo físico esperado para 3 vehículos con un tiempo de servicio medio cercano a 0.61 s, porque el sistema incorrecto permite más de tres entregas simultáneas. La versión corregida se mantuvo entre aproximadamente 4.75 y 4.96 solicitudes/s en la batería, cerca del límite dado por la flota.

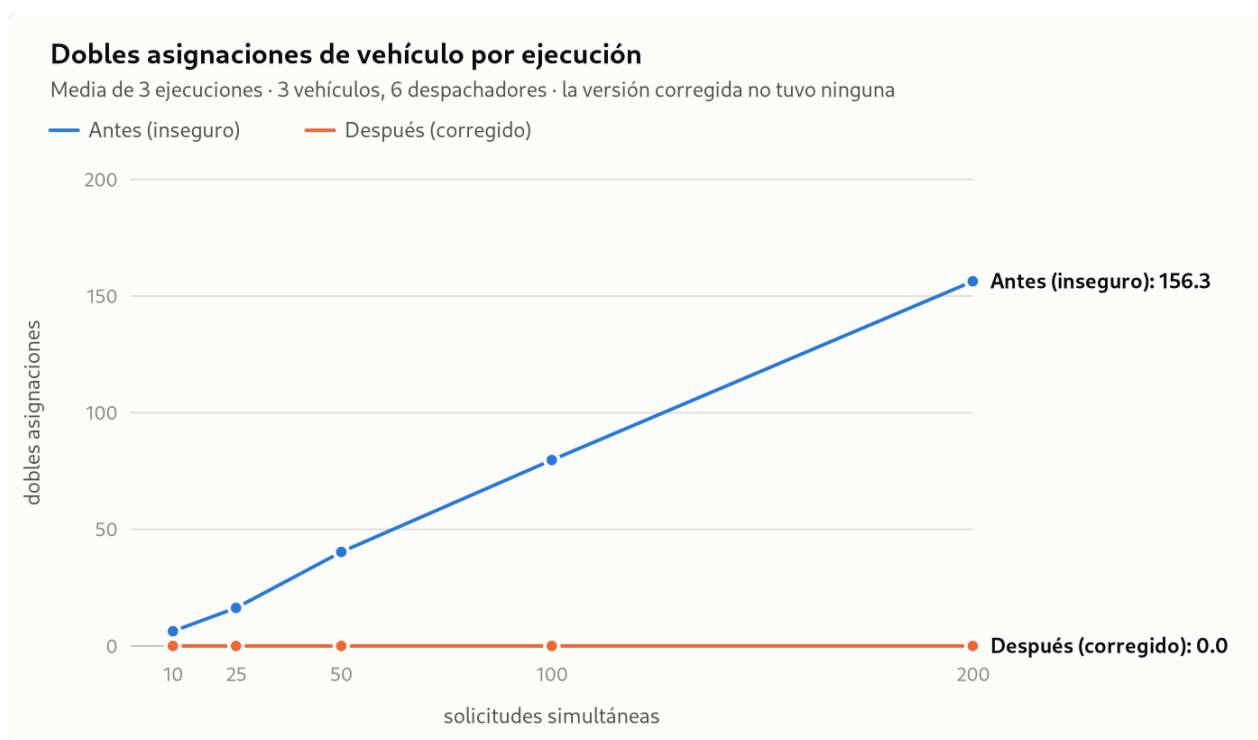


Figura 11. Dobles asignaciones observadas por carga. La serie corregida permanece en cero.

La comparación de interbloqueo también muestra un intercambio entre complejidad y tiempo de reacción. El orden global no requiere reintentos y el tiempo fue de 10.64 ± 0.13 s para 48 solicitudes. `timeout` presentó tiempos similares, pero con reintentos. La detección dejó ocurrir el ciclo y lo recuperó, con un tiempo de 12.19 ± 0.95 s para 48 solicitudes.



Figura 12. Interbloqueos no resueltos por estrategia. El modo sin orden no consigue completar la ejecución.

RESULTADOS

La evaluación final puede sintetizarse en cuatro resultados principales. En primer lugar, la sincronización convirtió la asignación de vehículos en una operación coherente: 0 dobles asignaciones en las 15 ejecuciones de la batería antes/después y 0 en las repeticiones adicionales de validación. El máximo de ocupación pasó de 6 operaciones simultáneas sobre 3 vehículos a 3 operaciones sobre 3 vehículos.

En segundo lugar, el análisis de interbloqueos confirmó que los órdenes opuestos crean un ciclo real y que las tres estrategias correctivas implementadas consiguen que el sistema termine. En la batería de seis ejecuciones por estrategia y dos cargas, el modo `sin_orden` se bloqueó en todas las ejecuciones, mientras que `orden`, `timeout` y `deteccion` no dejaron interbloqueos sin resolver.

En tercer lugar, la espera bloqueante mostró una diferencia de órdenes de magnitud respecto al sondeo. La configuración sin pausa consumió 25.62 s de CPU para realizar un trabajo que tomó prácticamente el mismo tiempo de pared que la configuración bloqueante, con 0.26 s de CPU. El número de cambios de contexto voluntarios también pasó de 1571 a más de 4.2 millones.

En cuarto lugar, el experimento CPU-bound mostró que el GIL impide obtener aceleración relevante con más hilos dentro de un solo proceso para código Python puro. La aceleración quedó alrededor de 1x, mientras que los procesos permitieron llegar a 1.76x con 4 procesos y 1.85x con 8 procesos en el equipo utilizado. Este resultado no significa que más procesos siempre sean mejores: la aceleración queda limitada por los dos núcleos físicos y el costo de planificación.

La métrica de memoria confirmó, además, que acotar el historial evita el crecimiento continuo del RSS. El modo sin límite llegó a 46.2 MB por trabajador, frente a 27.0 MB con 20 trazas retenidas.

La batería de 63 ejecuciones se construyó para aislar fenómenos. Cada celda tuvo tres repeticiones y se utilizaron entre 10 y 30 repeticiones adicionales en fenómenos intermitentes como carreras, inanición e interbloqueo. Las conclusiones son sólidas para el escenario evaluado, pero los tiempos absolutos dependen del equipo, su gobernador de frecuencia y la competencia con otros procesos.

CONCLUSIONES

1. Una aplicación concurrente puede completar todas sus solicitudes y, aun así, producir un estado incorrecto. En este proyecto, la versión insegura parecía más rápida porque permitía simultáneamente más “trabajo” del que físicamente permitía la flota. La instrumentación del estado y la auditoría de intervalos fueron necesarias para revelar el fallo.
2. La sección crítica correcta debe proteger la secuencia lógica completa. Proteger por separado la lectura y la escritura no habría eliminado la carrera; el mutex sobre buscar+marcar sí transforma la secuencia en una región indivisible.
3. El GIL no debe confundirse con sincronización de aplicación. Puede reducir la probabilidad de ciertos intercalados entre hilos, pero no garantiza la atomicidad de una secuencia lógica y no existe entre procesos [10].
4. Procesos e hilos resuelven problemas diferentes. Los procesos permiten paralelismo real de CPU en el CPython probado; los hilos son eficientes para representar muchas tareas que permanecen dormidas en `sleep`, colas o locks [8].
5. El interbloqueo aparece cuando coinciden exclusión mutua, retención y espera, no expropiación y espera circular. El orden global de adquisición rompe la espera circular; el `timeout` rompe la retención y espera; la detección con recuperación introduce expropiación coordinada [1].
6. Una espera bloqueante correctamente diseñada no sólo evita desperdiciar CPU, sino que deja al kernel gestionar el sueño y el despertar del hilo. En este proyecto, el tiempo total se mantuvo prácticamente constante mientras el consumo de CPU cayó de decenas de segundos de CPU a una fracción de segundo.
7. El análisis de memoria debe distinguir VSZ, RSS, PSS y memoria privada. El uso de PSS y `Private_Dirty` permitió diferenciar páginas compartidas tras `fork` de memoria privada escrita por cada trabajador [5].
8. Las mediciones de sistema operativo complementan la instrumentación propia de la aplicación. Cuando el log, el grafo de espera y las observaciones de `/proc` coinciden, la explicación de la causa del problema deja de depender de una única fuente.

Como limitaciones, el estudio se ejecutó en un único portátil con 2 núcleos físicos, frecuencia variable y tiempos de servicio simulados. Además, si un proceso muere reteniendo un lock de vehículo o andén, el

recurso puede quedar perdido; una implementación productiva debería incorporar recuperación de estado y mecanismos de lock robustos. Finalmente, la búsqueda de rutas por fuerza bruta fue escogida como carga

REFERENCIAS

- [1] E. G. Coffman, Jr., M. J. Elphick, and A. Shoshani, 'System deadlocks', ACM Computing Surveys, vol. 3, no. 2, pp. 67-78, 1971. [Online]. Available: <https://doi.org/10.1145/356586.356588>
- [2] M. Kerrisk and Linux man-pages project, 'proc-pid-task(5) · Linux manual page', 2026. [Online]. Available: https://man7.org/linux/man-pages/man5/proc_tid.5.html
- [3] M. Kerrisk and Linux man-pages project, 'proc-pid-stat(5) · Linux manual page', 2026. [Online]. Available: https://man7.org/linux/man-pages/man5/proc_pid_stat.5.html
- [4] M. Kerrisk and Linux man-pages project, 'proc-pid-status(5) · Linux manual page', 2026. [Online]. Available: https://man7.org/linux/man-pages/man5/proc_pid_status.5.html
- [5] M. Kerrisk and Linux man-pages project, 'proc-pid-smaps(5) · Linux manual page', 2026. [Online]. Available: https://man7.org/linux/man-pages/man5/proc_pid_smaps.5.html
- [6] Python Software Foundation, 'time · Time access and conversions · Python 3.14 documentation', 2026. [Online]. Available: <https://docs.python.org/3/library/time.html>
- [7] Python Software Foundation, 'resource · Resource usage information · Python 3.14 documentation', 2026. [Online]. Available: <https://docs.python.org/3/library/resource.html>
- [8] Python Software Foundation, 'threading · Thread-based parallelism · Python 3.14 documentation', 2026. [Online]. Available: <https://docs.python.org/3/library/threading.html>
- [9] Python Software Foundation, 'multiprocessing · Process-based parallelism · Python 3.14 documentation', 2026. [Online]. Available: <https://docs.python.org/3/library/multiprocessing.html>
- [10] Python Software Foundation, 'Thread states and the global interpreter lock · Python 3.14 documentation', 2026. [Online]. Available: <https://docs.python.org/3/c-api/threads.html>
- [11] M. Kerrisk and Linux man-pages project, 'futex(2) · Linux manual page', 2026. [Online]. Available: <https://man7.org/linux/man-pages/man2/futex.2.html>
- [12] E. Fonseca Jiménez, M. A. Delgado López, and J. M. Vianchá, 'Proyecto final de sistemas operativos: sistema de despacho y logística', GitHub repository, 2026. [Online]. Available: https://github.com/edwarfonseca/proyecto_final_so.git